

Building and Securing a REST API

MoMo SMS Transactions API - Project Report

1. Introduction to API Security

A REST API is the contract between a client (mobile app, web app, partner system) and the server that owns the data. Because anyone on the network can send a request, an unprotected API is the same as leaving a door open to the database. API security is the set of controls that decide **who** can call an endpoint, **what** they are allowed to do, and **how** the data is moved and stored safely.

For this project, the MoMo SMS data is sensitive: it contains phone numbers, names, amounts and balances. The API must therefore enforce at minimum:

- * **Authentication** - proving the caller is who they say they are.
- * **Authorisation** - deciding what that caller may read or change.
- * **Transport security** - protecting credentials in transit (HTTPS).
- * **Input validation** - refusing malformed or malicious payloads.

We implemented HTTP Basic Authentication as the first layer of defence. Section 4 explains why this is only a starting point and what we would adopt for production.

2. System Overview

The API is built in plain Python using `http.server` (no web framework). The data pipeline is:

modified_sms_v2.xml -> **dsa/parser.py** (XML -> list of dicts) -> **dsa/data_loader.py** (JSON persistence) -> **api/server.py** + **api/router.py** (HTTP server and routing, protected by **api/auth.py**).

On start-up the server loads the parsed JSON, builds an in-memory list and an id->transaction dictionary (used by the DSA experiment in Section 5), then begins serving on <http://localhost:8000>.

3. API Documentation

Base URL: <http://localhost:8000>

Authentication: HTTP Basic Auth. Credentials: admin:momo2024

Method	Path	Purpose
GET	/transactions	List every transaction
GET	/transactions/{id}	Retrieve one transaction by id
POST	/transactions	Create a new transaction
PUT	/transactions/{id}	Update fields of an existing transaction
DELETE	/transactions/{id}	Remove a transaction

3.1 GET /transactions - list all

```
curl -u admin:momo2024 http://localhost:8000/transactions
```

Response 200:

```
{
  "count": 1693,
  "transactions": [
    {
      "id": 1, "transaction_type": "received", "amount": 2000.0,
      "currency": "RWF", "sender": "Jane Smith",
      "timestamp": "2024-05-10T16:30:51"
    }
  ]
}
```

3.2 GET /transactions/{id} - retrieve one

```
curl -u admin:momo2024 http://localhost:8000/transactions/1
```

Response 200:

```
{
  "id": 1, "transaction_type": "received", "amount": 2000.0,
  "currency": "RWF", "sender": "Jane Smith",
  "sender_phone": "250XXXXXX013", "receiver": "Account Holder",
  "timestamp": "2024-05-10T16:30:51",
  "balance_after": 2000.0, "fee": 0.0,
  "transaction_id": "76662021700"
}
```

```
Error 404: {"error": "Transaction with id 1 not found"}
```

3.3 POST /transactions - create

Required fields: transaction_type, amount, timestamp.

```
curl -u admin:momo2024 -X POST http://localhost:8000/transactions \
-H 'Content-Type: application/json' \
-d '{"transaction_type": "payment", "amount": 500,
    "timestamp": "2026-05-30T10:00:00"}'
```

Response 201:

```
{
  "message": "Transaction created successfully",
  "transaction": {
    "id": 1694, "transaction_type": "payment",
    "amount": 500.0, "currency": "RWF"
  }
}
```

Error 400: {"error": "Missing required field: amount"}

3.4 PUT /transactions/{id} - update

```
curl -u admin:momo2024 -X PUT http://localhost:8000/transactions/1 \
-H 'Content-Type: application/json' -d '{"amount": 1500}'
```

Response 200:

```
{
  "message": "Transaction updated successfully",
  "transaction": {"id": 1, "amount": 1500, "transaction_type": "received"}
}
```

3.5 DELETE /transactions/{id} - delete

```
curl -u admin:momo2024 -X DELETE http://localhost:8000/transactions/1
```

Response 200:

```
{
  "message": "Transaction deleted",
  "transaction": {"id": 1, "transaction_type": "received", "amount": 2000.0}
}
```

3.6 Error Codes

Code	Meaning	When it is returned
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST
400	Bad Request	Missing required field, invalid JSON body
401	Unauthorized	Missing or wrong Basic Auth credentials
404	Not Found	Unknown id or unknown route

4. Reflection - Basic Auth Limitations

Basic Authentication is what the assignment asked us to implement and it is trivial to add: the client base64-encodes `user:pass` and sends it in the `Authorization` header. Our `api/auth.py` decodes that header on every request and compares it to the configured credentials, returning 401 on mismatch. However, Basic Auth has several real-world weaknesses:

1. **Base64 is encoding, not encryption.** The string `YWRTaw46bW9tbzIwMjQ=` decodes instantly to `admin:momo2024`. Anyone who captures a single request (packet sniffer on the LAN, compromised proxy, log file that records headers) has the password.
2. **Credentials travel on every call.** Each request re-sends the password, multiplying the chances of interception. Token-based schemes only send the secret once at login.
3. **No expiry and no revocation.** A leaked password is valid until a human rotates it. There is no concept of a session timeout, a logout button, or revoking a single device.
4. **Hard-coded credentials.** In the current build the username and password are constants in source code. Anyone with read access to the repository has the production credentials.
5. **No granular permissions.** Either a caller is *admin* or they are nobody. There is no read-only role, no per-endpoint scope, and no audit trail of who did what.

Stronger alternatives

JWT (JSON Web Tokens). The client logs in once with username and password; the server returns a signed token containing the user id, the role and an expiry (e.g. one hour). Every subsequent request carries the token in the `Authorization: Bearer ...` header. Benefits: the password is sent once, tokens expire automatically, the server can verify them statelessly using its signing key, and stolen tokens stop working after the TTL. This is the natural next step for our API.

OAuth 2.0. Delegates authentication to a trusted identity provider (Google, Microsoft, an in-house IdP). Our API never sees the user's password at all. OAuth adds scopes (e.g. `transactions:read`, `transactions:write`), refresh tokens, and central revocation. It is heavier to set up but is the right answer once third-party or mobile clients are involved.

Defence in depth, regardless of scheme. Always serve the API over HTTPS so headers cannot be sniffed; rate-limit login attempts to stop brute force; store secrets in environment variables, not source code; and log authentication failures for monitoring.

5. Data Structures & Algorithms

To compare search strategies we implemented two lookup methods in `dsa/search_algorithms.py` and timed them on the parsed dataset using `dsa/performance_test.py`.

5.1 Implementations

Linear search walks the list one element at a time and returns the first transaction whose id matches the target. Worst-case it inspects every record, giving $O(n)$ time.

Dictionary lookup builds an `id -> transaction` hash map once ($O(n)$ one-off build), after which each lookup is $O(1)$ average because Python hashes the key directly to a memory slot.

5.2 Experimental results

We loaded the full parsed dataset (1,693 transactions after filtering OTP messages), then looked up 20 different ids using each method and measured the elapsed time with `time.perf_counter()`.

Representative result from one run:

Method	Time for 20 lookups	Complexity
Linear Search	0.001842 s	$O(n)$
Dictionary Lookup	0.000012 s	$O(1)$ average
Speedup	~150x in favor of dict	

Exact numbers vary by machine and warm/cold cache, but the dictionary is consistently one to two orders of magnitude faster, and the gap widens as the dataset grows.

5.3 Reflection - why is dict lookup faster?

Linear search has to ask, for every element, "is this the one?" If our target id sits at position 1,600 the loop runs 1,600 comparisons. The average cost grows in direct proportion to the size of the list, which is the definition of $O(n)$.

A Python dictionary is a hash table. The key (the integer id) is fed through a hash function that produces a number; that number is mapped to a slot in an internal array, and the value is read directly from that slot. No matter whether the table holds 20 items or 20 million, the work is the same handful of CPU instructions, so each lookup is $O(1)$ on average.

5.4 Could we do even better?

For pure exact-match lookup by id, a hash map is already essentially optimal. However, real APIs have other access patterns where a different structure wins:

- * **Binary search** on a sorted list is $O(\log n)$. Useful when memory is tight and the data is already sorted, because it avoids the extra memory a hash table consumes.
- * **B-tree indexes** (used by every relational database) are $O(\log n)$ but excel at *range* queries, e.g. "all transactions between two dates" or "all amounts greater than 10,000 RWF" - queries a hash map cannot answer efficiently.
- * **Trie / inverted index** for text search. If users want to search by sender name prefix ("Jan...") or full-text search inside the SMS body, a trie or an inverted index (the structure behind Elasticsearch) is far better than scanning every row.

In short: choose the data structure to match the question you ask most often. For our REST API, the natural next move is to move from an in-memory dict to the SQLite database the team already designed, where indexes give us $O(\log n)$ lookups *and* persistence across restarts.

6. Testing & Validation

Every endpoint was exercised end-to-end using `curl`. Screenshots are included in the repository under `docs/screenshots/` and cover:

- * Successful authenticated GET (list and single).

- * 401 Unauthorized when credentials are missing or wrong.
- * Successful POST creating a new transaction (201).
- * Successful PUT updating an existing transaction (200).
- * Successful DELETE removing a transaction (200) and the follow-up GET returning 404.

7. Conclusion

The Croissant MoMo API meets the assignment's functional brief: SMS records are parsed from XML into structured JSON, exposed through a full CRUD REST interface, protected by Basic Authentication, and documented for other developers. The DSA experiment shows concretely why hash-based lookup is the right default for id-based access, and the security reflection lays out the path from Basic Auth to JWT or OAuth 2.0 for a production deployment. The next milestones - HTTPS, token expiry, and replacing the in-memory store with the team's SQLite schema - turn this prototype into something that could safely sit behind a real mobile or web client.